

Cloud Spanner：使用 Spanner Graph 演算法的 Graph Intelligence

來源：<https://codelabs.developers.google.com/spanner-graph-algorithms?hl=zh-tw>

步驟數：9

在本程式碼研究室中，我們將逐步從原始資料轉換為圖形表示法，並使用 Spanner Graph 演算法取得連結資料的智慧資訊。

目錄

1. 個案研究：智慧零售
2. 設定資料基礎
3. Spanner Graph 演算法簡介
4. 挑戰 1：影響力差距 (PageRank)
5. 挑戰 2：物流韌性 (中介中心度)
6. 挑戰 3：幽靈網路 (WCC)
7. 挑戰 4：行為雙生 (JaccardSimilarity)
8. 統一智慧
9. 恭喜和摘要

1. 個案研究：智慧零售

我們以**零售業客戶**為個案研究對象，該客戶的數位市集快速成長。傳統的顧客資料檢視畫面會顯示**顧客購買的商品**，但**不會顯示顧客之間的連結**，因此資訊有限。這項落差會導致錯失商機，詐欺行為也會增加。現在，他們轉向「**網路優先**」理念，除了交易資料外，也重視社群和物流連結。

待解決的核心業務難題

您面臨四項重大挑戰，需要瞭解顧客與物流的相互關係：

挑戰	問題	目標
影響差距	廣告範圍過於廣泛，投資報酬率偏低；目前無法找出真正的引領潮流者 (網紅)。	透過顧客連線網路，找出社群中的 影響者 。
物流韌性	供應鏈可能很脆弱 (考量到供應鏈在不同地理位置運作)。如果其中一個金鑰中樞發生故障，整個區域可能都會失去產品存取權。	找出 守門員 ，也就是將物流網路連結在一起的關鍵人物。
Ghost Networks	詐欺集團會使用虛假商家檔案和共用地址來協調竊取行為，並灌水評分。	揭露 孤立島嶼 ：與合法社群沒有關聯的超連結群組。
選擇悖論	目前的建議/推薦引擎相當基本、通用，而且經常遭到忽略 (例如「購買這項產品的顧客也買了...」)。	建立行為 雙胞胎 ，也就是根據類似的運送模式和社交圈提供建議。

將業務挑戰對應至技術策略 (列 → 關係)

在傳統資料庫中，資料會儲存在獨立的孤島中：顧客資料位於一個資料表，交易資料位於另一個資料表，運送資料則位於第三個資料表。SQL 非常適合回答「誰買了什麼？」這類問題，但難以回答與網路相關的問題。

為解決這些難題，技術策略是轉移這個觀點：

- 關係檢視畫面 (「什麼」)**：將每位顧客視為獨立資料列。如要找出顧客與朋友購買行為之間的關聯，需要進行多項複雜的「聯結」，而隨著網路擴大，速度會呈指數級下降。
- 圖表檢視畫面 (「如何」)**：將關係視為一等公民。我們在地圖上瀏覽，而不是搜尋清單。我們立即發現顧客 A 與顧客 B 連結，而顧客 B 的運送地點為 Z 地點。

技術策略

- 客戶無意捨棄現有的關聯式資料。
- 而是要在關係資料表上建立**屬性圖表**疊加層，揭露一直存在的隱藏影響和風險模式。
- On Graph 打算透過精密的**圖形演算法**解決業務挑戰。

深入瞭解相關規定

解決方案架構師得出結論，認為業務需求和技術策略需要採用多模型方法，並找出下列主要需求。

技術相關規定

- 可擴充的關聯式基礎
- 無須 ETL 即可建構圖表表示法
- 統一的查詢語言，可查詢關聯式資料和遍歷圖形資料。
- 可執行運算密集型圖形演算法，不會影響生產系統。
- 能夠擷取演算法結果，以進行後續分析 (最好是回到同一個資料庫)

Cloud Spanner 如何符合這些技術規定

我們選擇 Cloud Spanner 做為這項轉型的核心。這可讓客戶維持穩固的關聯式基礎，同時發掘深入的圖表洞察。

這不是要您在試算表和地圖之間做選擇，而是要**直接在試算表上建立地圖**。

以下簡要說明 Cloud Spanner 如何滿足技術需求等。

Cloud Spanner 技術規定檢查清單

- **零 ETL (無資料孤島)**：在傳統設定中，您必須將資料複製到獨立的圖形資料庫，這會造成延遲和安全風險。在 Spanner 中，圖表位於現有資料表上方。顧客完成購買後，圖表中的「邊」會立即存在。
- **交錯式效能**：Spanner 獨特的**交錯式資料表**功能，可將顧客的個人資料、好友和交易記錄儲存在同一位置。這項功能可將耗用大量網路資源的緩慢作業，轉換為快速的本機查詢。
- **盒外分析隔離 (Data Boost)**：客戶可以執行密集的圖形演算法 (例如 PageRank 或社群偵測)，完全不會拖慢結帳程序。使用 **Data Boost** 時，這些計算會在獨立的無伺服器資源上執行，因此「交易」端不會受到「智慧」探索的影響。
- **統一語言**：使用 **GoogleSQL** 產生標準報表，並使用 ISO **GQL (Graph Query Language)** 挖掘關係。您甚至可以在單一查詢中合併使用這兩者，同時找出顧客的電子郵件地址 (SQL) 和最具影響力的好友 (GQL)。

此外，Cloud Spanner 還提供可因應未來變化的技術架構

具有前瞻性

- 除了 Graph 之外，Spanner 還提供整合式資料平台，讓客戶日後能運用**全文搜尋**、執行即時 **AI 推論**、使用**向量搜尋**進行混合式推薦，以及在單一共存系統中建構真正的 **HTAP**。

步驟 2 · 約 0 分鐘

2. 設定資料基礎

完成商業案例後，我們現在要進入實作階段。在本節中，我們將定義資料架構、探討傳統關聯模型的限制，並介紹**屬性圖形**，這是我們發掘深入洞察資料的主要工具。

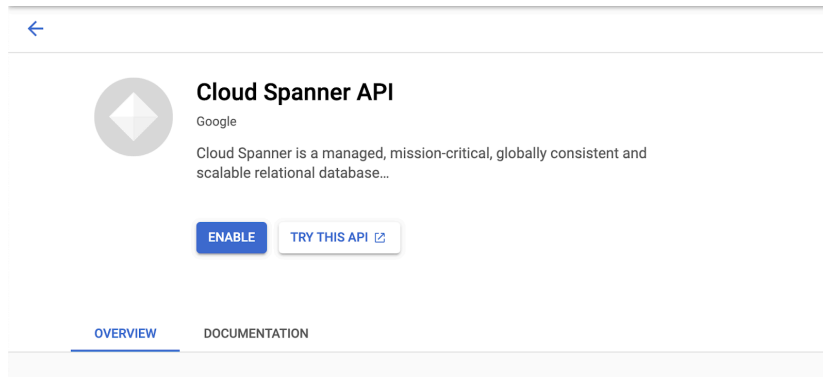
設定 Cloud Spanner Enterprise 執行個體

步驟 1：啟用 Cloud Spanner API

在 [Google Cloud 控制台](#) 中，按一下畫面左上角的「選單」圖示，開啟左側導覽選單。向下捲動並選取「Spanner」，或搜尋「Spanner」。

您現在應該會看到 Cloud Spanner 使用者介面。如果您使用的專案尚未啟用 Cloud Spanner API，系統會顯示對話方塊，要求您啟用該 API。如果已啟用 API，可以略過此步驟。

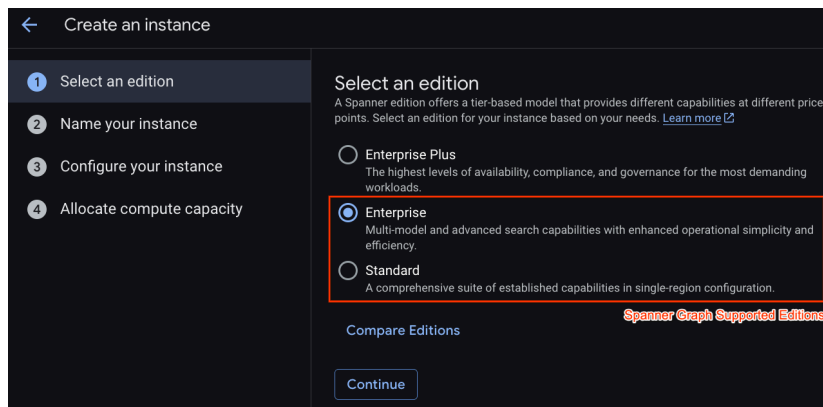
按一下「啟用」繼續操作：



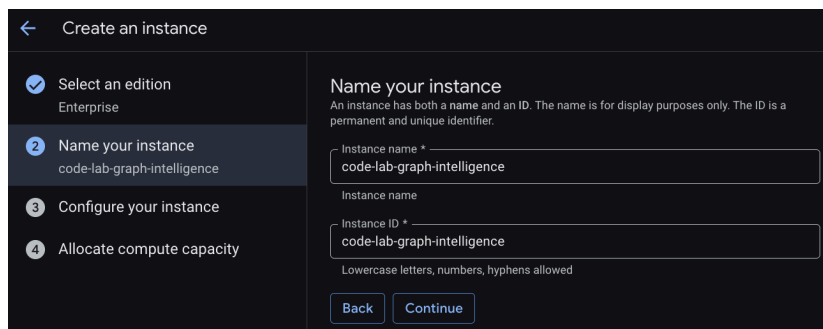
步驟 2：建立 Cloud Spanner 執行個體

首先，您要建立 Cloud Spanner 執行個體。在 UI 中，按一下「建立佈建執行個體」，建立新的執行個體。

在第一個步驟中，您必須選取版本。請注意，您也可以之後升級版本。如要使用多模型功能 (Spanner Graph)，可以選擇 **Enterprise** 版。



為執行個體命名



選取部署設定，然後選擇您想要的區域。

The screenshot shows the 'Create an instance' wizard with four steps: 1. Select an edition (Enterprise), 2. Name your instance (code-lab-graph-intelligence), 3. Configure your instance (us-central1 (Iowa)), and 4. Allocate compute capacity. The 'Choose a configuration' section explains that configuration determines node and data location. It offers three options: 'Regional' (99.99% availability SLA, lower write latencies within region), 'Dual-region' (99.999% availability SLA, from across two regions), and 'Multi-region' (99.999% availability SLA, from multi-geographic regions). The 'Dual-region' and 'Multi-region' options are highlighted with a red box and labeled 'Supported in Enterprise Plus'. A dropdown menu shows 'us-central1 (Iowa)' selected. A link to 'Compare region configurations' is also present.

您也可以比較各種設定選項。舉例來說，部署設定在所選區域的 3 個不同區域中，至少有 3 個讀取/寫入副本。也就是說，即使您選擇單一節點部署，也會透過 3 個讀取/寫入副本擁有 3 個副本。此外，即使使用區域部署設定，您也可以在部署拓撲中新增 R/O 副本，進一步擴充。

設定容量後，您可以從完整節點開始，並以節點為單位自動調度資源；也可以使用精細的執行個體（處理單元；1,000 個處理單元 = 1 個節點）。您也可以選擇設定執行個體的自動調度資源目標。以低延遲工作負載來說，建議您將區域執行個體的目標設為 65%，多區域執行個體則設為 45%。

The screenshot shows the 'Configure compute capacity' step of the 'Create an instance' wizard. It explains that compute capacity determines data throughput, queries per second (QPS), and storage limits. Under 'Select unit', 'Nodes' is selected (for large instances, 1 node equals 1,000 processing units), while 'Processing units (PUs)' is an alternative for small, granular instances. Under 'Choose a scaling mode', 'Manual allocation' is selected (for fixed resources and costs), while 'Autoscaling' is an alternative. A 'Quantity' input field shows '1' and a 'Nodes' button with a help icon is next to it.

步驟 3：建立資料庫

執行個體佈建完成後，請按一下「建立資料庫」，為其餘的程式碼研究室建立資料庫。

The screenshot shows the 'Create a database in code-lab-graph-intelligence' dialog. It prompts the user to 'Name your database' with a permanent name of at least two characters starting with a letter. The input field contains 'graph-algorithms'. Below the input, it notes that lowercase letters, numbers, hyphens, and underscores are allowed. The next section is 'Select database dialect', with options for 'Google Standard SQL' (selected) and 'PostgreSQL'.

建立關係基礎

首先，我們從儲存營運資料的核心資料表開始。在 Cloud Spanner 中，我們會使用交錯功能，將相關資料（例如顧客的友誼和交易）與顧客記錄直接共置於實體位置。確保高效存取和實體位置。

DDL：建立資料表

複製並執行下列區塊，建立關聯式結構定義：

```

-- NODE: Customer (Parent)
CREATE TABLE Customer (
  customer_id STRING(60) NOT NULL,
  customer_email STRING(32),

  -- Placeholder fields for Algorithm results
  pagerank_score FLOAT64,
  centrality_score FLOAT64,
  community_id INT64
) PRIMARY KEY(customer_id);

-- EDGE: CustomerFriendship (Interleaved in Customer)
CREATE TABLE CustomerFriendship (
  customer_id STRING(60) NOT NULL,
  friend_id STRING(60) NOT NULL,
  friendship_strength FLOAT64,
  created_at TIMESTAMP,
  CONSTRAINT FK_Friend FOREIGN KEY(friend_id) REFERENCES Customer(customer_id)
) PRIMARY KEY(customer_id, friend_id),
  INTERLEAVE IN PARENT Customer ON DELETE CASCADE;

-- NODE: Product
CREATE TABLE Product (
  product_id STRING(60) NOT NULL,
  product_name STRING(32),
  unit_price FLOAT64,
  pagerank_score FLOAT64
) PRIMARY KEY(product_id);

-- NODE: Shipping
CREATE TABLE Shipping (
  shipping_id STRING(60) NOT NULL,
  city STRING(32),
  country STRING(32)
) PRIMARY KEY(shipping_id);

-- EDGE: Transactions (Interleaved in Customer)
CREATE TABLE Transactions (
  customer_id STRING(60) NOT NULL,
  row_id STRING(36) DEFAULT (GENERATE_UUID()),
  product_id STRING(60) NOT NULL,
  shipping_id STRING(60) NOT NULL,
  transaction_date TIMESTAMP,
  amount FLOAT64,
  CONSTRAINT FK_Prod FOREIGN KEY(product_id) REFERENCES Product(product_id),
  CONSTRAINT FK_Ship FOREIGN KEY(shipping_id) REFERENCES Shipping(shipping_id)
) PRIMARY KEY(customer_id, row_id),
  INTERLEAVE IN PARENT Customer ON DELETE CASCADE;

```

為網路提供種子

準備好資料表後，我們必須填入定義客戶生態系統的使用者、產品和連結。

```
-- Populate Products & Shipping
INSERT INTO Product (product_id, product_name, unit_price) VALUES
('P1', 'Smartphone Pro', 999.00), ('P2', 'Wireless Earbuds', 150.00),
('P3', 'USB-C Cable', 25.00), ('P4', '4K Monitor', 450.00),
('P5', 'Ergonomic Chair', 300.00), ('P6', 'Desk Lamp', 45.00);

INSERT INTO Shipping (shipping_id, city, country) VALUES
('S1', 'New York', 'USA'), ('S2', 'London', 'UK'), ('S3', 'Tokyo', 'Japan'),
('S4', 'San Francisco', 'USA'), ('S5', 'Berlin', 'Germany');

-- Populate Customers
INSERT INTO Customer (customer_id, customer_email) VALUES
('C1', 'alice@example.com'), ('C2', 'bob@example.com'), ('C3', 'charlie@example.com'),
('C4', 'david@example.com'), ('C5', 'eve@example.com'), ('C6', 'frank@example.com'),
('C7', 'grace@example.com'), ('C8', 'heidi@example.com'), ('C9', 'ivan@example.com'),
('C10', 'judy@example.com'), ('C11', 'mallory@example.com'), ('C12', 'trent@example.com');

-- Populate Friendships
INSERT INTO CustomerFriendship (customer_id, friend_id, friendship_strength, created_at) VALUES
('C1', 'C2', 1.0, CURRENT_TIMESTAMP()), ('C1', 'C3', 1.0, CURRENT_TIMESTAMP()),
('C2', 'C1', 0.8, CURRENT_TIMESTAMP()), ('C3', 'C1', 0.9, CURRENT_TIMESTAMP()),
('C3', 'C4', 0.5, CURRENT_TIMESTAMP()), ('C4', 'C5', 0.5, CURRENT_TIMESTAMP()),
('C5', 'C6', 1.0, CURRENT_TIMESTAMP()), ('C5', 'C7', 0.8, CURRENT_TIMESTAMP()),
('C7', 'C8', 0.7, CURRENT_TIMESTAMP()), ('C8', 'C5', 0.6, CURRENT_TIMESTAMP()),
('C11', 'C1', 1.0, CURRENT_TIMESTAMP()), ('C11', 'C5', 1.0, CURRENT_TIMESTAMP()),
('C11', 'C7', 1.0, CURRENT_TIMESTAMP()), ('C11', 'C12', 0.5, CURRENT_TIMESTAMP()),
('C1', 'C11', 0.9, CURRENT_TIMESTAMP()), ('C5', 'C11', 0.9, CURRENT_TIMESTAMP()),
('C9', 'C10', 1.0, CURRENT_TIMESTAMP()), ('C10', 'C9', 1.0, CURRENT_TIMESTAMP());

-- Populate Transactions
INSERT INTO Transactions (customer_id, product_id, shipping_id, amount, transaction_date) VALUES
('C1', 'P1', 'S1', 999.00, CURRENT_TIMESTAMP()), ('C2', 'P1', 'S1', 999.00, CURRENT_TIMESTAMP()),
('C11', 'P4', 'S4', 450.00, CURRENT_TIMESTAMP()), ('C11', 'P5', 'S4', 300.00, CURRENT_TIMESTAMP()),
('C7', 'P5', 'S5', 300.00, CURRENT_TIMESTAMP()), ('C8', 'P6', 'S5', 45.00, CURRENT_TIMESTAMP()),
('C9', 'P1', 'S1', 999.00, CURRENT_TIMESTAMP()), ('C10', 'P1', 'S1', 999.00, CURRENT_TIMESTAMP());
```

關係挑戰

在介紹圖表之前，我們先來看看傳統 SQL 如何處理客戶的挑戰。執行這項查詢，找出消費金額高且朋友眾多的「社交消費者」。

```
SELECT
  c.customer_id,
  c.customer_email,
  SUM(t.amount) AS total_spent,
  COUNT(DISTINCT f.friend_id) AS friend_count
FROM Customer AS c
LEFT JOIN Transactions AS t ON c.customer_id = t.customer_id
LEFT JOIN CustomerFriendship AS f ON c.customer_id = f.customer_id
GROUP BY c.customer_id, c.customer_email
HAVING total_spent > 500
ORDER BY total_spent DESC;
```

關係方法的限制

在「網路優先」業務中，SQL 有幾個重大缺點：

- 「**聯結**」懲罰：如要找出朋友的朋友購買的商品，您必須聯結多個大型資料表。在 SQL 中，這需要彙整四個資料表 (Customer → Friendship → Transactions → Product)，隨著連線增加，運算成本也會隨之提高。
- 單一**跳躍的盲點**：關係查詢非常適合尋找直接連結 (單一跳躍)，但難以處理**遞移關係**。舉例來說，如果顧客 A 和顧客 B 共用地址，但顧客 B 只透過社群連結與顧客 C 建立關聯，您就必須在 SQL 中進行複雜的遞迴聯結，才能找出詐欺集團。
- 可讀性**：隨著客戶對連線的疑問越來越深入，SQL 查詢也變得越來越密集，難以維護。

透過屬性圖克服關係挑戰

為克服這些限制，我們定義了**屬性圖**。這會建立「疊加層」，讓我們將關係視為一等公民，不必將資料移出 Spanner。

DDL：建立屬性圖

這項 DDL 定義了我們的「節點」(實體) 和「邊緣」(關係)。在這個範例中，我們遵循結構化圖表，但 Spanner Graph 允許建立**無結構定義的圖表**，以實現彈性、快速的疊代開發，並處理不斷演進的資料模型，而不需持續變更 DDL (資料定義語言)。


```
CREATE OR REPLACE PROPERTY GRAPH RetailTransactionGraph
NODE TABLES (
  Customer KEY (customer_id),
  Product KEY (product_id),
  Shipping KEY (shipping_id)
)
EDGE TABLES (
  CustomerFriendship AS IsFriendsWith
    SOURCE KEY (customer_id) REFERENCES Customer (customer_id)
    DESTINATION KEY (friend_id) REFERENCES Customer (customer_id)
    LABEL IsFriendsWith,

  Transactions AS Purchased
    SOURCE KEY (customer_id) REFERENCES Customer (customer_id)
    DESTINATION KEY (product_id) REFERENCES Product (product_id)
    LABEL Purchased,

  Transactions AS LivesAt
    SOURCE KEY (customer_id) REFERENCES Customer (customer_id)
    DESTINATION KEY (shipping_id) REFERENCES Shipping (shipping_id)
    LABEL LivesAt
);
```

使用 GQL 瀏覽圖表

定義圖表後，我們就能使用 **Graph Query Language (GQL)**，以簡單易讀的語法執行多跳遍歷。

探索 1：協作探索

這項查詢會遍歷圖表，找出好友購買的產品，並做為推薦引擎的基礎。

```
GRAPH RetailTransactionGraph
MATCH (me:Customer)-[:IsFriendsWith]->(friend:Customer)-[:Purchased]->(p:Product)
WHERE me.customer_id = 'C1'
RETURN
  me.customer_id AS my_id,
  friend.customer_id AS friend_id,
  p.product_name AS recommendation
```

注意：語法 (節點)-[:EDGE]->(節點) 會以視覺化方式呈現遍歷。在 GQL 中，這個多重躍點跳轉是單一可讀取路徑，而不是四個資料表聯結。

探索 2：混合查詢 (關聯式 + 圖形)

您可以使用 GRAPH_TABLE 函式，將 GQL 模式內嵌在標準 SQL FROM 子句中。這項查詢會找出與朋友住在同一地點的顧客，也就是「菱形」模式比對。

```
SELECT *
FROM GRAPH_TABLE(RetailTransactionGraph
  MATCH (a:Customer)-[:IsFriendsWith]-(b:Customer),
        (a)-[:LivesAt]->(loc:Shipping),
        (b)-[:LivesAt]->(loc)
  RETURN a.customer_id AS user_A, b.customer_id AS user_B, loc.city
)
```

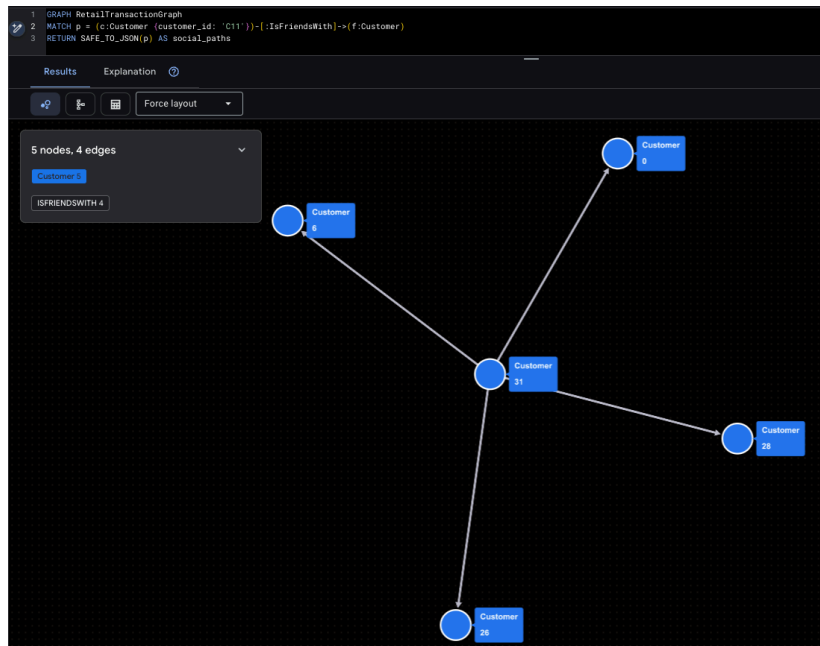
將顧客連結視覺化

最後，我們使用 GQL 將網路視覺化。這些查詢會將路徑結果包裝在 SAFE_TO_JSON 中，讓視覺化工具繪製節點和線條。

超級影響者視覺化

這項資料突顯了 Mallory (C11) 和她的直接社群觸及範圍。

```
GRAPH RetailTransactionGraph
MATCH p = (c:Customer {customer_id: 'C11'})-[:IsFriendsWith]->(f:Customer)
RETURN SAFE_TO_JSON(p) AS social_paths
```



以圖表呈現潛在的詐欺模式

這項查詢會找出「孤立叢集」(Ivan 和 Judy)：瞭解產品的運送地點。

```
GRAPH RetailTransactionGraph
MATCH p = (c:Customer)-[:Purchased]->(prod:Product),
      q = (c)-[:LivesAt]->(loc:Shipping)
WHERE c.customer_id IN ('C9', 'C10')
RETURN SAFE_TO_JSON(p) AS purchase_path, SAFE_TO_JSON(q) AS shipping_path
```

💡 我們發現了什麼？我們現在可以看到 Ivan 和 Judy 運送至相同地點，但他們在社群上與「The Customer」的「主體」沒有連結。在下一節中，我們將使用圖形演算法，以數學方式證明並自動偵測這些模式。

3. Spanner Graph 演算法簡介

為深入探索 Graph Intelligence，本節將說明 **Cloud Spanner Graph Algorithms**的技術架構和基本規則。瞭解這些原則是從簡單遍歷轉移到 PB 級關係分析的關鍵。

演算法組合

Cloud Spanner 目前支援 **14 種業界標準的圖形演算法**，分為四個功能群組，可解決各種業務問題：

類別	支援的演算法	業務用途
中心性	PageRank、個人化 PageRank、中介中心度、緊密度	找出影響者、中心和瓶頸。
社群	WCC、標籤傳播、集團搜尋、關聯性分群法	偵測詐欺集團、社群和孤島。
相似度	Jaccard、餘弦、共同鄰點、鄰點總數	支援推薦引擎和實體解析。
路徑尋找	集到集合的最短路徑、Google Analytics 路徑輔助程式	盡可能縮短物流和移動距離。

重要的結構定義和查詢注意事項

為確保圖形演算法能有效執行，Spanner Graph 必須遵守下列規則：

規定 1. 實體資料區域性 (交錯)

高效能圖形遍歷最關鍵的需求是**交錯**。這樣可確保邊緣資料實際儲存在與來源節點相同的伺服器分割區，盡量減少演算法執行期間的網路延遲。

- 規則：邊緣資料表必須交錯於來源節點資料表中。
- 轉送遍歷：將邊緣資料表交錯插入來源節點資料表，確保輸出連結的快取位置。
- 反向周遊：如要有效分析「傳入」連結，請使用外鍵自動建立支援索引，或在目的地資料表中建立交錯的次要索引。

⚠ 注意：如果未正確交錯邊緣資料表，將無法使用圖形演算法，標準遍歷效能也會大幅降低。請參閱「[圖形遍歷最佳做法](#)」一文中的 Spanner 結構定義。

規定 2：獨特標籤規定

參與屬性圖表的每個資料表都必須有專屬 ID。演算法會根據這些標籤，正確識別及載入需要分析的子圖。

- 規則：每個輸入資料表在屬性圖中都必須有專屬的識別標籤。
- 衝突：如果您打算在多個表格上執行演算法，就無法將單一標籤對應至多個表格。

邏輯	範例	結果
✖ 不佳	節點資料表 (人員標籤實體、帳戶標籤實體)	無效：演算法無法區分「人員」和「帳戶」。
✔ 良好	節點表格 (人員標籤客戶、帳戶標籤帳戶)	有效：每個實體都有專屬的唯一標籤。

規定 3：演算法查詢結構 (MATCH 子句)

呼叫演算法時，MATCH 子句會遵循比標準 GQL 查詢更嚴格的規則，確保執行引擎可以最佳化分析管道。

- 每個 MATCH 陳述式只能命名一個變數：每個 MATCH 陳述式只能命名一個變數。
- 沒有多節點模式：您無法在演算法呼叫適用的 MATCH 子句中，直接定義關係模式 (例如 (a)-[e]->(b))。
- 僅限常值篩選器：您可以使用 WHERE 子句篩選節點 (例如 WHERE a.id > 400)，但圖形演算法查詢目前**不支援**查詢參數 (@param)。

規定 4：RETURN 子句 (僅限純量)

演算法查詢中的 RETURN 子句是圖形世界和關聯世界之間的橋樑。只能傳回**純量**和**常數**。

- 規則：您無法傳回「圖形元素」(原始節點或邊緣物件)。
- 沒有轉換：您無法在 RETURN 陳述式本身中，對傳回的屬性執行數學運算或套用函式。

RETURN 子句限制

✔ 支援	✖ 不支援
RETURN node.id, score	傳回節點、分數 (無法傳回圖表元素)
RETURN PATH_LENGTH(p)	RETURN node.id + 1, score (No operations on properties)

RETURN node.name

RETURN JSON_OBJECT(node.id, score) (無函式)

要求 5. 資料完整性：消除懸空邊緣

如果邊緣指向圖表中不存在的目的地節點，就會發生「懸空邊緣」的情況。這會導致演算法執行失敗，因為圖表結構不一致。

- **解決方案：**使用參照限制 (外鍵) 和 ON DELETE CASCADE，維護圖表完整性。
- **查詢安全性：**呼叫演算法時，請務必確認所選邊緣參照的所有節點，也包含在 node_labels 引數中。

💡 **注意：**維持嚴格的參照完整性不僅能避免「懸空邊緣」錯誤，還能讓最佳化工具對圖的結構做出假設，進而提升整體查詢速度。

持續輸出：匯出資料選項

由於圖形演算法需要大量運算資源，因此會使用 `EXPORT DATA` 陳述式，以擴大執行模式執行。這項功能會運用 **Data Boost**，使用獨立的無伺服器運算資源，避免生產交易發生任何延遲。

選項 1：將資料保留回 Cloud Spanner

如要將結果直接推回資料表 (例如儲存 PageRank 分數)，請使用 format = 'CLOUD_SPANNER'。

- **update_ignore_all**：只更新目標資料表中已有的鍵。
- **upsert_ignore_all**：更新現有資料列，或在缺少鍵時插入新資料列。

```
EXPORT DATA OPTIONS (  
  format = 'CLOUD_SPANNER',  
  table = 'Customer',  
  write_mode = 'update_ignore_all'  
) AS  
GRAPH RetailTransactionGraph  
CALL PageRank(...)  
RETURN node.customer_id, score;
```

方法 2：將結果保存至 Google Cloud Storage (GCS)

如要進行大規模離線分析，您可以匯出為 **CSV**、**Avro** 或 **Parquet** 格式的 GCS。

- **萬用字元：**使用 uri => 'gs://bucket/file_*.csv' 啟用輸出資料分割，讓 Spanner 能平行寫入多個檔案，處理大量資料集。
- **壓縮：**支援 GZIP、SNAPPY 和 ZSTD，可盡量降低儲存費用。

```
EXPORT DATA OPTIONS (  
  uri = 'gs://bucket/pagerank_*.csv',  
  format = 'CSV',  
  overwrite = true  
) AS  
GRAPH RetailTransactionGraph  
CALL PageRank(...)  
RETURN node.customer_id, score;
```

⚠️ 重要注意事項和警示

不支援查詢參數：目前圖形演算法查詢不支援查詢參數 (@param)。所有引數 (例如阻尼係數或節點標籤) 都必須是查詢字串中的常值。

純量回傳作業：您無法在演算法呼叫的 RETURN 子句中直接執行數學運算或套用函式 (例如，不支援 RETURN node.id + 1)。

無伺服器計費：Spanner Graph 演算法採用無伺服器計費模式。系統只會根據演算法特定執行期間耗用的無伺服器處理單元 (SPU) 向您收費。

4. 挑戰 1：影響力差距 (PageRank)

在本節中，我們將探討客戶的第一個業務障礙：**影響力差距**。我們將從基本的「人氣競賽」，轉向以數學為基礎的真實社群影響力地圖。

問題陳述：客戶的行銷團隊遇到問題。他們在廣泛的廣告上投入數百萬美元，但回報率卻不斷下降，因為他們無法找出「社群巨星」，也就是那些罕見的個人，他們的代言會影響整個網路。

為解決這個問題，我們需要依影響力為顧客排名。

關聯解決方案 (程度中心性)

在標準資料庫中，尋找網紅最簡單的方法就是計算追蹤者人數 (這項指標稱為「中心度」)。

執行這項查詢，找出最「熱門」的使用者：

```
SELECT
    friend_id AS customer_id,
    COUNT(*) AS follower_count
FROM CustomerFriendship
GROUP BY friend_id
ORDER BY follower_count DESC;
```

customer_id	follower_count
C1	3
C5	3
C11	2
C7	2
C10	1
C12	1
C2	1
C3	1
C4	1
C6	1
C8	1
C9	1

⚠️ 數量與品質

這個 SQL 方法會將所有追蹤者視為同等。系統無法區分使用者追蹤的對象是 100 個閒置帳戶，還是 10 位高價值「VIP」網紅。

在我們的資料中，**Alice (C1)** 和 **Eve (C5)** 的追蹤者人數相同。關聯式 SQL 顯示兩者相同，**但如您所見，圖表顯示的結果大相逕庭。**

圖表智慧 (PageRank)

為了找出真正的領導者，我們使用 **PageRank**。這項演算法與早期網頁搜尋採用的演算法相同，會根據連入連結的**數量**和**品質**，評估節點的重要性。

- **隨機衝浪者模型：**PageRank 會模擬使用者在圖表中移動，**阻尼係數** (預設為 0.85) 代表繼續點擊的機率；否則，他們會「傳送」至隨機節點。
- **關聯性：**來自有影響力人士 (例如 Mallory) 的連結，價值遠高於來自沒有其他連結的人。

我們會執行 PageRank 演算法，並使用 EXPORT DATA 將結果直接儲存到 pagerank_score 資料欄。

```
EXPORT DATA OPTIONS (
  format = 'CLOUD_SPANNER',
  table = 'Customer',
  write_mode = 'update_ignore_all' -- Updates existing rows
) AS
GRAPH RetailTransactionGraph
CALL PageRank(
  node_labels => ['Customer'],      -- Target our Customer nodes
  edge_labels => ['IsFriendsWith'], -- Analyze the social ties
  damping_factor => 0.85,           -- Standard decay
  max_iterations => 10              -- Higher iterations for better precision
)
YIELD node, score
RETURN node.customer_id, score as pagerank_score;
```

使用 PageRank 的「影響力」資訊主頁

現在分數已儲存，讓我們比較「之前」(追蹤者人數) 和「之後」(PageRank 分數)。

```
-- Note that Higher PageRank score means more influential
SELECT
  c.customer_id,
  c.customer_email,
  count_query.follower_count,
  c.pagerank_score
FROM Customer c
JOIN (
  SELECT friend_id, COUNT(*) AS follower_count
  FROM CustomerFriendship GROUP BY friend_id
) AS count_query ON c.customer_id = count_query.friend_id
ORDER BY c.pagerank_score DESC;
```

customer_id	customer_email	follower_count	pagerank_score
C5	eve@example.com	3	0.158392489
C10	judy@example.com	1	0.1093561724
C9	ivan@example.com	1	0.1093561724
C1	alice@example.com	3	0.1000888124
C8	heidi@example.com	1	0.09759821743
C11	mallory@example.com	2	0.09466411918
C7	grace@example.com	2	0.08016719669
C6	frank@example.com	1	0.06022448093
C2	bob@example.com	1	0.0547891818
C3	charlie@example.com	1	0.0547891818
C12	trent@example.com	1	0.04029225558
C4	david@example.com	1	0.04028172791

分析：誰是真正的超級巨星？

分析輸出內容後，您現在可以做出三項重要的行銷發現：

- 💡 **隱藏的寶石 (Eve, C5)**：雖然 Eve 和 Alice 的追蹤者人數相同，但 **Eve 的 PageRank 顯著較高**。這是因為 Eve 屬於「互惠迴圈」，她的朋友也具有影響力。
- 💡 **迴圈效應 (Ivan 和 Judy, C9/C10)**：Ivan 和 Judy 各只有一位追蹤者，但 PageRank 比 Alice 高。由於他們只追蹤彼此，因此會在獨立迴圈中來回傳遞重要性。
- ⚠️ 我們需要調查「詐欺集團」部分的「迴圈效應」。
- 💡 **重質不重量的勝利 (Mallory, C11)**：Mallory 的追蹤者人數少於 Alice，但相較於觸及人數，她的分數較高。PageRank 認為 Mallory 與合適的對象有連結，因此是品牌大使的高效率目標。

商務重點

行銷團隊不必再盲目地傳送電子郵件給所有追蹤者超過五人的使用者，現在可以專注於 **pagerank_score** 最高的使用者。這些人才是真正的「社群巨星」，有能力在整個市場中引發系統性病毒式傳播。

現在，讓我們試著找出負責維持客戶物流網路運作的守門員。

5. 挑戰 2：物流韌性 (中介中心度)

本節將說明**物流韌性**。我們將不再以「數量」衡量成功，而是找出維持網路連線的關鍵「守門員」。

⚠️ 如果主要中樞或特定「橋接器」連線發生故障，整個子網路可能無法存取產品。我們可以透過交易量追蹤這項資訊，但重要節點可能「隱藏」在網路結構中，因此找出這些節點非常重要。

關係解決方案 (以量為準的分析)

在標準關聯設定中，「重要」運送中心通常是指處理最多訂單或產生最多收益的中心。

執行這項查詢，找出交易次數最多的「熱門」中心：

```
-- Identify "Critical" hubs by transaction volume
SELECT
    s.city,
    s.country,
    COUNT(t.row_id) AS transaction_count,
    SUM(t.amount) AS total_revenue
FROM Shipping s
JOIN Transactions t ON s.shipping_id = t.shipping_id
GROUP BY s.city, s.country
ORDER BY transaction_count DESC;
```

city	country	transaction_count	total_revenue
紐約	USA	4	3996
柏林	德國	2	345
舊金山	USA	2	750

⚠️ 數量 != 嚴重程度

這種做法會產生盲點。小型區域中心 (例如柏林，S5) 可能只會處理幾筆訂單，但如果這是整個區域的**唯一**路徑，一旦發生故障，造成的災難性影響遠比有多個備援替代方案的大型中心 (例如紐約，S1) 嚴重。**SQL 會匯總資料，但無法「查看」使用者路徑之間的實體結構。**

為解決不符問題，我們會使用 `IsFriendsWith` 和 `LivesAt` 邊緣。這項功能可將分析從交易中心轉變為包含社群檢查。

圖表智慧 (中介中心度)

如要找出真正的瓶頸，我們使用「中介中心度」。這個演算法會量化節點在圖表中所有其他節點配對之間最短路徑上，充當「橋樑」的頻率。高分代表真正的把關者，他們掌控商品或資訊的流動。

⚠️ 物流韌性

物流穩定性與社會信任息息相關。我們使用 **IsFriendsWith** (*social*) 和 **LivesAt** (*logistics*) 邊緣分析圖表，因為「系統性守門員」是將不同社群群組連結至實體運送中心的中樞。如果這些「橋樑」節點發生故障，推薦內容的社群流程和商品的實體流程都會中斷。

執行並保留中介中心度

我們將使用 `EXPORT DATA` 執行演算法，並將分數儲存至 `centrality_score` 資料欄。我們使用**Data Boost**，確保這項繁重的「最短路徑」計算對客戶的即時作業幾乎沒有影響。


```
EXPORT DATA OPTIONS (
  format = 'CLOUD_SPANNER',
  table = 'Customer',
  write_mode = 'update_ignore_all'
) AS
GRAPH RetailTransactionGraph
CALL BetweennessCentrality(
  -- We include both Customer and Shipping nodes for a full ecosystem view
  node_labels => ['Customer', 'Shipping'],
  -- We factor in social ties AND physical shipping locations
  edge_labels => ['IsFriendsWith', 'LivesAt'],
  num_source_nodes => 100
)
YIELD node, score
-- We only persist scores for Customers; Shipping node results are safely ignored
RETURN node.customer_id, score as centrality_score;
```

分析：找出「隱藏的瓶頸」

現在，我們比較結構性風險 (centrality_score) 與交易量 (order_count)，找出客戶領導階層應留意的節點。

```
SELECT
  c.customer_id,
  c.customer_email,
  c.centricity_score,
  count_query.order_count
FROM Customer c
LEFT JOIN (
  SELECT customer_id, COUNT(*) AS order_count
  FROM Transactions GROUP BY customer_id
) AS count_query ON c.customer_id = count_query.customer_id
ORDER BY c.centricity_score DESC;
```

customer_id	customer_email	centricity_score	order_count
C11	mallory@example.com	44.5	2
C1	alice@example.com	35.5	1
C5	eve@example.com	35.5	
C7	grace@example.com	12	1
C8	heidi@example.com	10	1
C3	charlie@example.com	6	
C4	david@example.com	3.5	
C10	judy@example.com	0	1
C12	trent@example.com	0	
C2	bob@example.com	0	1
C6	frank@example.com	0	
C9	ivan@example.com	0	1

分析這些結果後，客戶有三項驚人發現：

💡 **超級守門員 (Mallory, C11)**：Mallory 的風險分數最高，結構分析顯示，她是主要的**全球錨點**，可將不同的社群與實體物流基礎架構連結起來。如果她的帳戶或相關聯的集線器遭到入侵，會比任何其他節點中斷更多客戶和出貨集線器之間的流程。

💡 **社群閘道 (Alice、C1 和 Eve、C5)**：Alice 和 Eve 的風險分數完全相同，都是 35.5 分。這些節點會做為鏡像節點，各自做為所屬社群叢集的出口點，用來存取全球網路。

💡 **結構價值與交易價值**：標準 SQL 報表會優先顯示高支出者，完全忽略結構性瓶頸。

業務重點

現在，客戶可以根據**多模式結構風險**，優先處理物流備援和安全通訊協定。**Mallory、Alice 和 Eve** 是守門員，必須受到保護，才能確保物流網路穩定運作。

現在，我們來試著找出詐欺島。

步驟 6 · 約 0 分鐘

6. 挑戰 3：幽靈網路 (WCC)

在本節中，我們將解決第三個業務障礙：「幽靈網路」。我們將從簡單的「熱點」偵測，轉向使用社群偵測功能，揭露複雜且孤立的詐欺集團。問題在於，惡意行為人會建立虛假個人資料，共用運送地址或在封閉迴路中互動，以協調竊盜行為並灌水產品評分。但這些社群通常與正當的「客戶」社群完全隔離。

如要解決這個問題，我們需要公開這些「孤立島嶼」。

關聯式解決方案 (共用 ID 搜尋)

如果沒有圖形演算法，尋找詐欺行為的標準做法是找出共用資料的「熱點」，例如多位顧客運送至完全相同的地址。

執行這項查詢，找出共用運送地點的連結顧客：

```
SELECT
  shipping_id,
  COUNT(DISTINCT customer_id) AS customer_count,
  ARRAY_AGG(customer_id) AS linked_customers
FROM Transactions
GROUP BY shipping_id
HAVING customer_count > 1;
```

shipping_id	customer_count	linked_customers
S1	4	["C1","C10","C2","C9"]
S5	2	["C7","C8"]

⚠ 遞移性失明

這項查詢只會找出直接的「1-hop」連結。如果客戶 A 和客戶 B 共用地址，且客戶 B 是客戶 C (使用不同地址) 的「好友」，標準 SQL 無法輕易將客戶 A 連結至客戶 C，視為同一群組。

如要找出詐欺網路，我們需要瞭解遞移可到達性。

圖表智慧 (弱連通元件)

如要找出這些環的完整範圍，我們使用弱連結元件 (WCC)。WCC 是一種叢集演算法，可找出任意兩個節點之間存在路徑的節點集，無論邊緣的方向為何。

- **可抵達區域**：這會有效地將圖表劃分為「島嶼」或「可抵達區域」。
- **統一實體檢視畫面**：同時分析社群關係 (IsFriendsWith) 和物流關係 (LivesAt)，將分散的個人資料歸入單一的「影響叢集」。

執行及保留 WCC

我們會執行 WCC 演算法，並將結果儲存到 community_id 欄。我們使用 **Data Boost**，確保這項深入的觸及率分析是在獨立的運算資源上進行。

```
EXPORT DATA OPTIONS (
  format = 'CLOUD_SPANNER',
  table = 'Customer',
  write_mode = 'update_ignore_all'
) AS
GRAPH RetailTransactionGraph
CALL WeaklyConnectedComponents(
  node_labels => ['Customer', 'Shipping'],
  edge_labels => ['IsFriendsWith', 'LivesAt']
)
YIELD node, cluster
-- node.customer_id will be NULL for Shipping nodes;
-- EXPORT DATA will safely ignore those rows.
RETURN node.customer_id, cluster AS community_id;
```

分析：詐欺集團

現在，讓我們執行驗證查詢，看看隔離的社群。正當使用者通常屬於「大陸」，而詐欺者通常會困在小「島嶼」上。

```
SELECT
  community_id,
  COUNT(*) AS member_count,
  ARRAY_AGG(customer_email) AS members
FROM Customer
GROUP BY community_id
ORDER BY member_count ASC;
```

community_id	member_count	成員
1	2	["judy@example.com","ivan@example.com"]
0	10	["alice@example.com","mallory@example.com","trent@example.com","bob@example.com","charlie@example.com","david@example.com","eve

執行這項社群偵測作業後，您就能找出重大異常狀況：

 **社群 1：隔離的「詐欺集團」：**這個群組只包含兩個成員：**judy@example.com (C10)** 和 **ivan@example.com (C9)**。

WCC 已標示這兩位客戶與其他 10 位客戶完全隔離。即使這些網頁的 PageRank 分數可能很高 (因為它們會形成迴圈)，但路徑數為零，無法連往客戶網路的其餘部分。

 **牽連犯罪：**如果 Judy 持有遭竊信用卡，系統現在會知道 Ivan 是她特定叢集/社群的一份子，因此應立即標記以供審查

業務重點

客戶現在可以自動執行安全回應。他們不必手動追蹤個別帳戶，只要編寫簡單的規則即可：「如果 community_id 的成員少於三人，請標記整個群組，以進行手動 KYC (瞭解您的客戶) 審查」。

揭露詐欺集團後，我們就能解決「行為雙生」問題。

步驟 7 · 約 0 分鐘

7. 挑戰 4：行為雙生 (JaccardSimilarity)

在最後一項挑戰中，我們將解決第四個障礙：「選擇悖論」/「行為雙生」。我們將從一般「經常一起購買」清單，改為根據行為「指紋」提供高度個人化建議。

目前為顧客提供的產品建議過於籠統。向每位顧客推薦熱門的 USB 傳輸線很安全，但並不貼心。客戶希望建立「行為雙胞胎」建議，找出具有獨特運送模式和社交圈的顧客，並建議高精度度相符的產品。

為解決這個問題，我們需要計算使用者之間的「鄰近程度」。

關係解決方案 (絕對重疊)

在標準關聯式設定中，您可能會尋找與參考使用者 (例如 **Alice (C1)**) 運送地點相同的人。

執行下列查詢，找出 **Alice** 的地理位置鄰居：

```
SELECT
  t2.customer_id AS similar_customer,
  COUNT(DISTINCT t1.shipping_id) AS shared_locations
FROM Transactions t1
JOIN Transactions t2 ON t1.shipping_id = t2.shipping_id
WHERE t1.customer_id = 'C1' AND t2.customer_id != 'C1'
GROUP BY similar_customer
ORDER BY shared_locations DESC;
```

similar_customer	shared_locations
C2	1
C10	1
C9	1

⚠️ 「聯集」問題

這項查詢會顯示哪些人分享位置資訊，但不會顯示他們獨一無二的相似程度。如果 Alice 運送至 100 個地點，Ivan 只運送至 1 個地點，那麼他們「相似」的程度，就比不上兩人都只運送至同一個地點。SQL 無法有效計算這個比率 (交集與聯集)。

圖形智慧 (Jaccard 相似度)

如要找出真正的行為雙胞胎，我們會使用 **Jaccard 相似度**。這個演算法會將共用鄰居數 (交集) 除以不重複鄰居總數 (聯集)，計算出標準化分數 (0.0 到 1.0)。

這裡的「行為雙胞胎」定義不只是共用運送地址，透過分析**實體足跡** (`LivesAt`) 和**社群生態系統** (`IsFriendsWith`) 的交集，我們可以找出具有相同生活型態和社群影響力的使用者，進而提供更準確的產品建議。

首先建立對應表

由於相似度是**成對關係** (客戶 A 類似於客戶 B)，因此我們會在 `Customer` 中建立專用的交錯式資料表，用來儲存這些對應關係。

```
CREATE TABLE CustomerSimilarity (
  customer_id STRING(60) NOT NULL, -- Renamed from source_id to match Parent PK
  target_id STRING(60) NOT NULL,
  similarity_score FLOAT64,
  CONSTRAINT FK_SourceCustomer FOREIGN KEY(customer_id) REFERENCES Customer(customer_id),
  CONSTRAINT FK_TargetCustomer FOREIGN KEY(target_id) REFERENCES Customer(customer_id)
) PRIMARY KEY(customer_id, target_id),
  INTERLEAVE IN PARENT Customer ON DELETE CASCADE;
```

現在執行 Jaccard 相似度

現在要執行演算法。**注意**：這項查詢包含常見的「防護措施」課程。如果您只選取「顧客」節點，但使用 `LivesAt` 邊緣 (指向「運送」節點)，查詢會失敗並顯示「懸空邊緣」。如要修正這個問題，我們必須同時加入節點標籤。

```

EXPORT DATA OPTIONS (
  format = 'CLOUD_SPANNER',
  table = 'CustomerSimilarity',
  write_mode = 'upsert_ignore_all'
) AS
GRAPH RetailTransactionGraph
CALL JaccardSimilarity(
  node_labels => ['Customer', 'Shipping'], -- Added Shipping to avoid dangling edges
  edge_labels => ['LivesAt', 'IsFriendsWith'], -- Use both logistics and social edges for holistic similarity
  source_nodes => ARRAY(
    SELECT s FROM GRAPH_TABLE(RetailTransactionGraph
      MATCH (s:Customer {customer_id: 'C1'})
      RETURN s)
  ),
  target_nodes => ARRAY(
    SELECT t FROM GRAPH_TABLE(RetailTransactionGraph
      MATCH (t:Customer)
      WHERE t.customer_id != 'C1'
      RETURN t)
  )
)
YIELD source_node, target_node, similarity
RETURN
  source_node.customer_id AS customer_id,
  target_node.customer_id AS target_id,
  similarity AS similarity_score;

```

⚠ 附註：

`node_labels => ['Customer', 'Shipping']` – we include **'Shipping'** to prevent dangling edges. 也就是說，在 `node_labels` 中加入「Shipping」可確保 `LivesAt` 邊緣的目的節點載入演算法的工作區，防止邊緣懸空，進而滿足子圖一致性規則

分析：「行為雙胞胎」檢查

分析工作完成後，我們會執行驗證查詢。加入新的對應表（`CustomerSimilarity`）和原始 `Customer` 中繼資料後，我們就能確切瞭解 Alice 的「行為雙胞胎」是誰。

執行這項查詢，檢查 Alice 的相似度排名：

```

SELECT
  c.customer_email AS peer_email,
  s.similarity_score,
  c.community_id,
  c.pagerank_score
FROM CustomerSimilarity s
JOIN Customer c ON s.target_id = c.customer_id
WHERE s.customer_id = 'C1'
ORDER BY s.similarity_score DESC;

```

peer_email	similarity_score	community_id	pagerank_score
judy@example.com	0.200000003	1	0.1093561724
bob@example.com	0.200000003	0	0.0547891818
ivan@example.com	0.200000003	1	0.1093561724
eve@example.com	0.1666666716	0	0.158392489
mallory@example.com	0	0	0.09466411918
trent@example.com	0	0	0.04029225558
charlie@example.com	0	0	0.0547891818
david@example.com	0	0	0.04028172791
frank@example.com	0	0	0.06022448093

grace@example.com	0	0	0.08016719669
heidi@example.com	0	0	0.09759821743

如何解讀結果：

💡 **0.20 領導者：**如您所見，Bob、Judy 和 Ivan 的分數約為 0.20，因此名列前茅。由於我們現在會同時考量社交圈和出貨中心，因此 Jaccard 公式 (交集 / 聯集) 計算出的重疊程度會比單純考量物流更細緻。

💡 **社群分割：**我們發現 Judy 和 Ivan 位於獨立的社群 1 島嶼。這個分數會提供「接近程度」，但社群 ID 會提供「信任度」。

⚠️ **「零」群組：**Mallory、Trent 和 Charlie 等使用者的分數為 0 分。這項數學運算結果證實，他們與 Alice 完全沒有共同點，既不是出貨中心，也沒有共同朋友，因此最不可能成為個人化推薦的對象。

現在，讓我們嘗試建立最終的整合式智慧檢視畫面。

8. 統一智慧

現在我們將從個別技術工作轉向統一智慧。我們將交易資料與所有四種圖形演算法混合，提供清楚且可執行的洞察資料。

報表 1：整合式智慧

Spanner 這類多模態資料庫的強大之處，在於能夠在單一要求中，將關聯式支出資料與圖形衍生影響力、風險和相似度分數合併。這項查詢會將每位顧客歸類到特定商家角色。

執行「整合式智慧」查詢，查看完整生態系統：

```
SELECT
  c.customer_id,
  c.customer_email,
  -- Transactional Data (Relational)
  COALESCE(t.total_spend, 0) AS spend,
  -- Graph Intelligence Data (Algorithms)
  c.pagerank_score AS influence,
  c.centrality_score AS bottleneck_risk,
  c.community_id,
  -- Persona Categorization Logic
  CASE
    WHEN c.community_id = 1 THEN '🔴 HIGH RISK: Isolated Fraud Ring'
    WHEN c.centrality_score > 25 THEN '🟡 CRITICAL: Network Bridge'
    WHEN c.pagerank_score > 0.08 AND t.total_spend > 500 THEN '🌟 VIP: Influential Spender'
    WHEN c.pagerank_score > 0.08 THEN '📢 SOCIAL: High-Reach Influencer'
    WHEN sim.similarity_to_alice = 1.0 AND c.community_id != 0 THEN '⚠️ WARNING: Identity Anomaly'
    ELSE '🟢 STANDARD: Active Customer'
  END AS business_persona
FROM Customer c
LEFT JOIN (
  -- Aggregate total spend per customer
  SELECT customer_id, SUM(amount) AS total_spend
  FROM Transactions GROUP BY customer_id
) t ON c.customer_id = t.customer_id
LEFT JOIN (
  -- Pull similarity relative to our reference user 'C1'
  SELECT target_id, similarity_score AS similarity_to_alice
  FROM CustomerSimilarity WHERE customer_id = 'C1'
) sim ON c.customer_id = sim.target_id
ORDER BY c.centrality_score DESC, c.pagerank_score DESC;
```

customer_id	customer_email	支出	影響力	bottleneck_risk	community_id	business_persona
C11	mallory@example.com	750	0.09466411918	44.5	0	🟡 嚴重：網路橋接器
C5	eve@example.com	0	0.158392489	35.5	0	🟡 嚴重：網路橋接器
C1	alice@example.com	999	0.1000888124	35.5	0	🟡 嚴重：網路橋接器
C7	grace@example.com	300	0.08016719669	12	0	📢 SOCIAL：高觸及率的影響者
C8	heidi@example.com	45	0.09759821743	10	0	📢 SOCIAL：高觸及率的影響者
C3	charlie@example.com	0	0.0547891818	6	0	🟢 標準：活躍客戶
C4	david@example.com	0	0.04028172791	3.5	0	🟢 標準：活躍客戶
C10	judy@example.com	999	0.1093561724	0	1	🔴 高風險：孤立的詐欺集團
C9	ivan@example.com	999	0.1093561724	0	1	🔴 高風險：孤立的詐欺集團
C6	frank@example.com	0	0.06022448093	0	0	🟢 標準：活躍客戶
C2	bob@example.com	999	0.0547891818	0	0	🟢 標準：活躍客戶
C12	trent@example.com	0	0.04029225558	0	0	🟢 標準：活躍客戶

結合這些數學觀點後，我們就能從「誰花最多錢」轉向「誰最重要」。整合式資訊主頁會整合關聯交易資料和多模態圖形智慧，將您的生態系統分類為三種明確且可執行的目標對象。

「關鍵網路橋接器」(韌性)

Mallory (C11)、Eve (C5) 和 Alice (C1) 等節點會標示為異常，因為這些節點的 `bottleneck_risk` (中介中心性) >25。

- 結構錨點：Mallory 的風險分數最高，為 **44.5**，因此是整個網路的主要閘道。
- 零支出悖論：Eve (C5) 的訂單數為零，但結構上不可或缺，風險分數為 **35.5**。標準 SQL 會完全忽略她，但圖表智慧功能會顯示她是整個子社群的重要橋梁。
- 高價值閘道：Alice (C1) 與 Eve 並列第一，都是 **35.5** 分，證明高消費族群也能成為重要的結構錨點。

💡 商家洞察：即使 Eve 的支出為零，在結構上仍不可或缺。如果她流失，你可能會失去與整個子社群的主要橋樑。

「社群超級巨星」(觸及)

Heidi (C8) 和 Grace (C7) 的 PageRank 分數較高，因此被視為高觸及率的影響者。

💡 業務洞察：網紅是您進行病毒式行銷時應採用的對象。他們在群組內人脈廣闊，可有效率地傳播資訊。

「Isolated Fraud Ring」(異常狀況)

Judy (C10) 和 Ivan (C9) 遭到標記，因為他們屬於隔離的 `community_id 1`

💡 業務洞察：儘管 Judy (C10) 和 Ivan (C9) 的支出很高 (999)，但他們是「孤島」。這是詐欺集團的典型特徵，他們會使用共用的代發貨中心，但與正當的社交網絡保持距離。

從業務洞察到策略行動

Persona	主要指標	商家洞察	策略行動
🌐 網路橋接器	高中心性	結構錨點：Eve (C5) 和 Mallory (C11) 維持網路運作。	留存率：保護這些守門員，避免社群分裂。
📰 社群媒體超級巨星	高 PageRank	病毒式引擎：在自己的社交圈中，Heidi (C8) 這類使用者觸及人數最多。	行銷：用於高影響力的推薦和品牌大使計畫。
🔴 詐欺風險	Isolated WCC	幽靈網路：Judy (C10) 和 Ivan (C9) 是高消費族群，但住在「島嶼」上。	安全性：立即進行人工 KYC 審查，這些是典型的詐欺特徵。
🟢 標準使用者	平衡分數	健康的核心：大部分的網路，包括 David (C4) 等「本機」橋接器。	成長：套用標準個人化廣告和「行為相似目標」最佳化建議。

重點

- 發掘隱藏價值：找出重要使用者，例如伊芙 (C5)，她的支出為 **0**，但對聯播網的存續至關重要，重要性甚至高於高支出者。
- 證明全球真相：修正當地假設，顯示 **David (C4)** 是次要的當地連結，而 **Mallory (C11)** 才是真正的全球閘道。
- 自動降低風險：立即區分正當的「中國大陸」顧客和可疑的「孤立島嶼」顧客。

報表 2：身分異常報表

現在您需要瞭解詐騙集團是否「模仿」正當帳戶。我們可以找出行為相似度 **100%** 但社群連結為零的使用者，藉此解決這個問題。

執行這項查詢，標示出潛在的「身分識別異常」：

```
SELECT
  s.target_id AS suspect_id,
  c.customer_email,
  s.similarity_score AS behavioral_overlap,
  c.community_id AS social_group
FROM CustomerSimilarity s
JOIN Customer c ON s.target_id = c.customer_id
WHERE s.customer_id = 'C1' -- Reference Alice (Legitimate)
  AND s.similarity_score > 0.15
  AND c.community_id != 0 -- Filter for social strangers
ORDER BY s.similarity_score DESC;
```

「找出異常狀況」報表會提供重要資訊。我們將行為類似正當顧客但缺乏社群連結的使用者隔離，從猜測轉為數學上的確定性。

suspect_id	customer_email	behavioral_overlap	social_group
C10	judy@example.com	0.200000003	1
C9	ivan@example.com	0.200000003	1

分析結果

我們將**相似度 (Jaccard)** 與**社群偵測 (WCC)** 整合，揭露傳統交易資料無法發現的隱藏風險。

- 「行為雙胞胎」(鄰近)：系統會標記 **Judy (C10)** 和 **Ivan (C9)** 等節點，因為相對於 Alice (C1)，這些節點的 Jaccard 相似度分數為 **0.20**。

💡 **商家洞察**：這些帳戶共用 Alice 的實體物流足跡 (運送至同一個紐約中心)，但社群重疊程度為零。在標準關聯式資料庫中，他們可能只是高價值顧客；但在這裡，我們發現他們會模仿她的習慣，但並非她社交圈的一份子。

- 隔離行為：Judy (C10) 和 Ivan (C9) 分組到隔離的 **community_id 1**，而 Alice 屬於「Mainland」社交群組 (社群 0)。

💡 **商家洞察**：雖然這些帳戶模仿了 Alice 的物流資訊，但完全沒有與她的人脈建立社交連結，這是一大警訊。這種結構性區隔是他們不屬於受信任使用者群組的重要指標。

- 詐欺標記：這份報表會找出行為重疊程度高 (超過 0.9)，但仍與主要網路保持社交連結的使用者。

💡 **商家洞察**：這可能表示詐欺帳戶會模仿正當足跡，以避免遭到偵測。

步驟 9 · 約 0 分鐘

9. 恭喜和摘要

本實驗室說明 Cloud Spanner 如何將關聯式資料庫轉變為多模型強大資料庫。我們將圖形智慧應用於**客戶**，從靜態資料轉為可執行的業務策略。

Spanner 多模型優勢

- **統一架構**：Spanner 可讓您維持穩固的關聯基礎，同時即時「疊加」屬性圖表，以挖掘關係，完全不會有 ETL 的風險和延遲。
- **獨立分析隔離**：運用 **Data Boost**，您可以在獨立的無伺服器運算資源上執行 PageRank 或 WCC 等耗用大量記憶體演算法，確保生產結帳效能不受影響。
- **交錯式效能**：Spanner 獨特的交錯式設計可確保節點及其關係位於同一位置，將複雜的全球遍歷轉換為高速的本機查詢。

顯示「隱藏的寶石」和異常狀況

- **找出結構價值**：圖表演算法 (例如中介中心度) 揭露了「隱藏橋樑」，這些橋樑**支出為零**，但對網路的復原能力而言，可能比支出最高的顧客更重要。
- **揭露行為模仿**：結合 **Jaccard 相似度**和弱連結元件，找出「社交陌生人」。這些帳戶看起來像是合法消費者，但數學證明這些帳戶是獨立的詐欺集團。
- **全域與本機真相**：手動 SQL 分析可以找出橋樑，但全域演算法可以找出網路的主要守門員。

讓資料變得智慧且可做為行動依據

- **人物角色導向策略**：我們成功將資料列轉換為關係，並透過執行演算法解決四個業務問題，分別是：**網路橋樑**、**社群巨星**、**詐欺風險**和**標準使用者**。

💡 透過彌合「列」和「關係」之間的差距，您已開發出「網路優先」解決方案，可建構真正智慧的系統，發掘資料中隱藏的關係。